

APPENDIX B

THE PIDGIN PRIMER

This appendix provides a description of PIDGIN, PARSIFAL's grammar input notation.

PIDGIN is a formal language that reads much like English, although its syntax is very restrictive and completely artificial. There are several advantages to using an English-like formal language as a notation for writing grammars. By putting the grammar into an English-like notation, the grammar can be intrinsically self-documenting. Because it reads much like English, PIDGIN code should be highly perspicuous both to those with computational backgrounds and to those linguists and psychologists not familiar with LISP syntax; in short, it attempts to be a *Perspicuous, Intrinsically Documented Grammar Input Notation*. The language also conveniently enforces a strict set of constraints on the form of legal grammar rules. By the simple expedient of not including the standard LISP primitives of CAR, CDR, CONS, etc. in the grammar language, we can strictly limit the sorts of operations that can be used within the grammar.

PIDGIN is internally converted to LISP by a parser which is based on Pratt's notion of top-down operator precedence parsing [Pratt 73], which provides a powerful framework for constructing grammars for simple token-based deterministic computer languages. Indeed, much of the code which underlies the PIDGIN parser derives from Pratt's implementation of his CGOL language [Pratt 76]. It is worthy of mention that the implementation of CGOL, a language designed as an alternate external representation for general LISP code, was very easily adapted for PIDGIN, a language with a very different purpose and external appearance.

It should also be noted that the cost of using this non-LISP syntax is not very high. The top-down operator precedence parser is very fast and efficient, and by using the read-macro facility of MACLISP [Moon 74], one can read in PIDGIN code through the normal LISP reader with little difficulty. Indeed, it

is easy to get the MACLISP compiler to directly compile PIDGIN code into machine language instructions.

Each grammar rule in PIDGIN is of the general form:

```
{Rule <name> (Priority: <numeric priority>) in <packet list>
  <pattern> --> <action>}
```

Each rule is delimited by "{ }", i.e. by curly brackets, and includes a rule name, an optional priority, a list of the packets the rule is part of, and a pattern and an action. Rule and packet names can be any legal LISP atom containing at least one non-numeric character, but not spaces, tabs, carriage returns or any of the special characters " | [] / { } () ' ; , = % ! . ". These special characters delimit names, and need not be separated from them. A priority can be any positive integer between 1, the highest priority, and 1000, the lowest. If a rule specifies no explicit priority, it is assigned a default priority of 10. The packet list consists of either one name or else a series of names separated by commas. Note that a rule can be in more than one packet; it will be active if any packet it is in is active.¹

The following sections present the core of PIDGIN syntax. I will first present the syntax of patterns and the subset of PIDGIN commands legal in patterns, then the remainder of basic PIDGIN commands which are of use primarily in rule actions.

Patterns

Patterns are written in PIDGIN as a list of descriptions, each description enclosed in square brackets. The first three descriptions in a pattern are taken by default to refer to the constituents in the first, second, and third buffer positions, referred to in PIDGIN as 1st, 2nd, and 3rd respectively. The following pattern is a typical example of a simple pattern, which looks for an auxiliary verb followed by an NP:

```
[* is verb, auxverb] [* is NP]
```

1. This facility is not used in the current grammar.

This pattern will be satisfied if 1st, the constituent in the first buffer position, has the features *verb* and *auxverb*, and 2nd, the constituent in the second buffer position, has the feature *NP*. This pattern does not impose any conditions on 3rd; indeed, there need be no constituent in the third buffer cell for this pattern to match. The predicate "is" tests that the node named by its first argument is labelled with all the features in the feature list that follows the predicate. "*" within a description refers to the node that the description refers to.

Since descriptions so often test for the positive presence of sets of features, PIDGIN allows the notation "= <feature list>" in patterns as a shorthand for "* is <feature list>", where <feature list> is either a single feature, or a list of features separated by commas.

A description that refers to C, the current active node, begins with the label "** C" followed by a semi-colon. The following pattern will match if 1st has the feature *NP* and C has the features *S* and *ynquest* (yes-no-question):

[=NP] [**C; =s, ynquest]

PIDGIN provides several other predicates besides "is" which test for the presence or absence of sets of features of a node. These predicates are:

<node> is (all of) <feature list>
 <node> is any of <feature list>
 <node> is not (all of) <feature list>
 <node> is none of <feature list>

OR

<node> is not any of <feature list>

Note that the last two predicates above are synonymous constructs; PIDGIN tries to provide synonymous constructs whenever there is more than one simple natural way to express an operation.

In this figure and below, I will use angle brackets, "<" and ">", to indicate meta-syntactic strings. Words surrounded by parentheses are optional; they may be used or not, at the option

of the grammar writer. Also, PIDGIN totally ignores the tokens "the", "a" and "an"; these can be used freely in the grammar to increase readability and will be used in examples below without further comment.

These predicates can be combined with logical operators to form arbitrary Boolean feature tests. A fairly complex test might look like:

(B.1) 1st is not an auxverb or 3rd is a verb.

The logical operations allowed, and their corresponding PIDGIN expressions are:

conjunction:	<pred1> and <pred2> and ... and <predn> OR (in patterns) <pred1>; <pred2>;; <predn>
disjunction:	<pred1> or <pred2> or ... or <predn>
negation:	it isn't true that <pred>

A precedence ordering is imposed on these operators to allow proper scoping in most cases without parentheses. The precedence order is (where "X > Y" means that X has higher precedence than Y):

"it isn't true that" > "and" > "or" > ";;".

Thus, for example (B.2a) below has as its meaning (B.2b):

(B.2a) <p1> or <p2> and it isn't true that <p3>; <p5> or <p6>
(B.2b) {<p1> ∨ [<p2> ∧ ¬<p3>]} ∧ {<p5> ∨ <p6>}

The idea behind the use of semi-colon in patterns as a conjunction operator is that semi-colons can be used to separate the set of predications that make up each pattern description; all of these predicates must be true to satisfy the description. Note that the "it isn't true that" operator takes narrow scope. Thus (B.3) is equivalent to (B.1) above:

(B.3) it isn't true that 1st is an auxverb or 3rd is a verb.

If the other sense is desired, parentheses can be used to specify the proper scopes. Thus, to get the wide scope negation sense of (B.3) one would say

It Isn't true that (1st is an auxverb or 3rd is a verb).

While parentheses *can* be used to change the "natural" scopes of these operators, there really shouldn't be much need to do so. This is especially true given two operators with different scopes for logical conjunction in patterns and the wide range of feature testing predicates.

To complete the subset of PIDGIN used in pattern descriptions, the phrase below is a tree traversing operator which can be used to name the sons of a node:

<type> of <fathernode>

This phrase returns as its value the node attached to <fathernode> as <type>, or the rightmost node attached as <type> if there is more than one. This construct is right associative, e.g. the two phrases below are equivalent:

the NOUN of the NP of C
(the NOUN of (the NP of C))

These expressions start with the current active node and then down through the NP attached to that node to the NOUN under the NP node.

To access the current cyclic node, PIDGIN allows the phrases²

the S above * OR the NP above *
the current S OR the current NP

One last primitive useful in rule patterns is the primitive "t", the always true predicate. A description made up of only this primitive will match any node, but such a description will force

2. Actually, the S above * is implemented as a general tree climbing operator, although it is not allowed to be used as such formally; in this sense, the current LISP implementation of the parser is more powerful than the specs presented here imply. This operator is used as a general tree climbing operation in the semantic tests used to determine PP attachment in the current grammar. These tests, which fall outside of the range of the research described in the rest of this document, seem to need to scan most of the active node stack. In many ways, PP attachment seems to be different than the sorts of phenomena discussed in this document.

the pattern matcher to wait until some constituent fills the corresponding buffer position before the pattern will match. This is important because each of the parameters 1st, 2nd and 3rd has a value within the action of a rule only if the pattern of that rule has a description for the corresponding buffer position.

A few example patterns should help to clarify the use of this syntax.

The following is the pattern of the rule **PASSIVE** discussed in Chapter 5:

[** c; * is a VP; the AUX of the S above * is passive; * is not np-preposed]

This pattern looks at the current active node (i.e. C), the current cyclic node (the S above *), and some of their daughters. It will match if C is a verb phrase, if the auxiliary attached to the current S has the feature *passive*, and C itself does not have the feature *np-preposed* (which is used to indicate that "passive-shift" has already taken place). Note that PIDGIN completely ignores the "a" in "* is a VP"; this phrase merely tests to see if * has the feature VP.

The following pattern will match against the first node in the buffer if it has the feature *PP*, i.e. it is a prepositional phrase, and it immediately dominates a preposition marked with the feature **OF*, indicating that the root of the word under the prep node is "of":

[=pp; the prep of * is *of] [t]

This pattern will match any node in the second buffer position, but it will not match until there is some node in the second buffer position. Unless 2nd was referenced by this pattern, the action of the rule of which this is the pattern would be unable to examine this constituent. Note, by the way, that case is entirely ignored in PIDGIN.

The rule which starts infinitive complements without overt subjects (phrases like "to buy a car") after verbs which take such a complement, e.g. the verb "want" as in "I want to buy a car", has the following pattern:

[=*to] [=tnsless] [** c; the verb of * is subj-less-inf-comp]

This pattern will match if 1st is the word "to", 2nd is a tenseless verb, and C, implicitly assumed here to be a VP, has the feature *subj-less-inf-comp*, indicating that it takes subject-less infinitive complements.

Actions On Nodes

The basic action commands in PIDGIN operate on nodes; these commands create them, attach them, and pop them from the active node stack.

The basic command for creating new parse nodes is:

Create a (new) <type> node (labelled <feature set>)

which creates a new node of type <type>, makes it the current active node, and optionally labels it with the features in <feature set>. An alternative way of saying the same thing is:

a new <type> node (labelled <feature set>)

This construct for node creation returns the name of the newly created node as its value. It can thus be used in composition with other primitive operations such as attachment. Note that the token "new" is obligatory here.

One very useful grammatical operation is replacing "deleted" lexical items, as discussed in Chapter 5. To facilitate this sort of operation, PIDGIN provides a composite command which inserts a node dominating a particular lexical item into the buffer in a specified position:

Insert the word '<lexical item>' (into the buffer) (before <position>)

This command inserts a node dominating <lexical item> into the buffer before <position>, which can be any of 1st, 2nd, or 3rd. If <position> is not specified, then the node is inserted into the buffer before 1st. Thus, the PIDGIN code to convert "all the" into "all of the", assuming "all" is 1st and "of" 2nd would be:

Insert the word 'of' into the buffer before 2nd.

Later rules will then process the input as if "of" appeared in the original input string.

To attach <node> to <fathernode>, PIDGIN provides the command:

Attach <node> to <fathernode> as <type>

For the time being, <type> can be considered to be a redundant specification of the type of the new daughter node.

As a special case, the semantics of the form

Attach a new <type> node ... to C as <type>

are defined to mean: Create a new <type> node, attach it to the old current active node, and *then* make the new node the current active node. This definition solves by *fiat* the problem of whether "C" refers to the newly created node or the old current active node.

Once the parser is finished with a current active node, it must pop the active node stack. The "drop" command does this:

Drop c (into the buffer).

If C is attached to some other node, this command simply pops C from the active node stack. If C is not attached, this command pops it off the active node stack and inserts it into the buffer in the 1st buffer position, shifting the other buffered constituents one to the right. Note that the optional phrase "into the buffer" carries no significance whatever to the PIDGIN reader. It can be used for the sake of perspicuity and self-documentation, if the grammar writer knows that the node about to be dropped is not attached to another node.

One last operation on nodes that is most useful is the ability to test if a node has a daughter of a given type. For instance, a grammar of English must check to see if a NP has a determiner when checking the person and number of the NP. If there is a determiner and it is "a" or "an", then the head noun must also be singular if the NP is to be grammatical. In PIDGIN, given that the NP in question is C, the predicate of this test will be simply

there is a det of c and it is *a.

The "there is" predicate is of form

there is <node>

where <node> is most likely to be a complex specification for a node. The predicate is true if <node> exists and false otherwise. Since <node> is most likely a long expression, PIDGIN provides the convenience that the token "it" refers to the node found by the last "there is" predicate evaluated. If the last "there is" test failed, then "it" has as its value the LISP atom NIL. All predicates applied to NIL are false.³

Feature Manipulating Primitives

PIDGIN provides many primitives for manipulating features. The most basic of these add, and, for the moment, remove given features.

The command

Label <node> with (the feature(s)) <feature set>

adds the features in <feature set> to the features of <node>. Note, by the way, that the two clauses below are exactly equivalent:

Label a new S node with the features decl, major.

Create a new S node labelled decl, major.

Although I believe that the constraint that features never be removed can be enforced, PIDGIN provides the command

Remove (the feature(s)) <feature set> from <node>

as a utility for grammar debugging and development.

A very common operation is "transferring" some general class of features from a daughter node to its parent, labelling the parent with those features of the daughter node that are in a given set. To facilitate this operation, PIDGIN provides the construct:

3. This combination of "there is" and "it" was borrowed from CGOL.

(B.4) Transfer (the feature(s)) <feature set> from <node1> to <node2>.

This construct adds to the features of <node2> the intersection of the features of <node1> and <feature set>. Thus, for example, one can shift the singular-plural features *ns*, noun singular, and *npl*, noun plural, and the 1st, 2nd, and 3rd person features *n1p*, *n2p*, and *n3p* from a NOUN to its parent NP (in this case assuming the NP is 1st) with the command:

Transfer the features *ns*, *npl*, *n1p*, *n2p*, *n3p* from the NOUN of 1st to 1st.

Besides specifying feature lists literally, several operations exist for manipulating feature lists themselves. These operations allow the grammar to access the feature set of a particular node, and to take the intersection of two separate feature lists.

To directly access the features of a node, PIDGIN provides the phrase:

the features of <node>.

which returns the feature set of <node>. To set the features of a node, one can use the command:

Set the features of <node> to <feature set>.

To take the intersection of two feature sets, one uses the phrase:

the meet of <feature set> and <feature set>.

PIDGIN allows the use of "meet" and "feature" phrases everywhere it allows literal feature sets. Thus, (B.4) above can be seen as an abbreviation for

Label <node2> with the meet of the features of <node1> and <feature set>.

Conditionals

While PIDGIN provides no control primitives allowing iteration within rules, or calls to subroutines, it does allow conditional expressions. The basic form of a conditional is

If <Boolean expression> then <complex action> (else <complex action>).

The <Boolean expression> can be made up of any logical composition of predicates. The <complex actions> can be either 1) primitive actions, 2) conditionals (allowing for nested conditionals) or 3) a sequence of complex actions separated by semi-colons i.e.

<complex action₁>; <complex action₂>;...;<complex action_n>.

Such a complex action sequence (a LISP "PROGN") is executed, as would be expected, from left to right (or, if you prefer, top to bottom).

It is important to note that semi-colons have a *different meaning* in actions than in patterns. Semi-colons in patterns have the sense of logical conjunction. In actions, they separate the primitive actions of complex action sequences and *do not* have this logical sense. They cannot be used in actions to make up complex predicates.

Given that there can be nested conditionals, i.e. conditionals of the form

(B.5) If <pred1> then if <pred2> then <action1> else <action2>,

there must be rules for scoping the nested conditionals. As is commonly done, "else" clauses are scoped by the nearest "if", thus the phrase (B.5) above is equivalent to:

If <pred1> then (if <pred2> then <action1> else <action2>).

Similarly, complex action sequences are scoped to the nearest conditional. Thus, (B.6a) below means the same as (B.6b) but not (B.6c).

(B.6a) If <p1> then <act1>; If <p2> then <act3>; <act4>.

(B.6b) If <p1> then <act1>; (If <p2> then <act3>; <act4>).

(B.6c) If <p1> then <act1>; (If <p2> then <act3>); <act4>.

Parentheses, as used in (B.6a)-(B.6c), are legal here as elsewhere to specify scope. However, their use is discouraged; the philosophy of PIDGIN is to encourage the user to write code that reads like English.

Parser Control

In this section I will present the PIDGIN primitives used to control the parser itself. Two of these primitives activate and deactivate packets, a third is used to indicate that the parse is finished, and a fourth provides a very limited inter-rule control facility.

The two PIDGIN commands to activate and deactivate packets are

Activate <packet list>

Deactivate <packet list>.

In each of these two commands <packet list> may be a single packet name or a list of packet names separated by commas.

A grammar rule action can signal the grammar interpreter that the parse has been completed with the command

The parse is finished.

Executing this command sets a flag which the grammar interpreter checks after each rule action is finished. If the flag is set, the interpreter returns control to whatever higher level process originally called the grammar interpreter. Note that the parse does not terminate immediately upon execution of this command, but rather after the rule it is part of has finished.

A rule can determine its own successor, circumventing the pattern matching process, with the command

Run <rule> next.

This command sets a flag which the grammar interpreter checks after the completion of each rule to the name of the rule to be run next. If the flag is set to a rule name, the grammar

interpreter executes the action of that rule immediately instead of pattern matching to find out what rule to run next. Only those parameters of 1st, 2nd, and 3rd which had values within the action of the calling rule have values in the action of the called rule; i.e. it is as if the action of <rule> had been appended to the action which executed this phrase.

Note that this mechanism is essentially a device for abbreviating grammar rules and allowing the grammar writer to capture generalizations. This mechanism allows the grammar writer to abbreviate several rules that end with the same code by making the common code a separate rule and having each of the original rules run it after they are done. Empirically, it seems that there is usually independent motivation for the common code to be a rule in its own right, so forcing the common code to be a separate rule rather than using some sort of macro expansion mechanism seems to be appropriate. In either case, since this mechanism is purely abbreviatory, it adds no real power to the form of grammar rules.

Comments

PIDGIN allows grammar writers to add comments anywhere within rule actions and patterns. Comments in PIDGIN consist of any text surrounded by "%"s. (To include a "%" in a comment, type two of them.) It is perfectly legal for a single comment to run on for several lines.

Actions

Grammar rule actions are made up of a sequence of instructions, where each instruction is either one of the primitive commands discussed above or an "if..then...else..." conditional expression. Each instruction in an action ends with a period. These instructions are executed sequentially from first to last; each action is thus equivalent to a LISP "PROGN".

How to Throw In the Kitchen Sink

In a natural language front end for a large application-oriented system, it is often convenient to cross the line between syntax and semantics and embed some low level semantic processing in the grammar itself; system development also often requires linking arbitrary pieces of LISP code into the parser. To allow this to be done where it may be useful, PIDGIN includes primitives which set and fetch registers which are associated with specified parse nodes, and includes a notation which allows arbitrary LISP code to be embedded in PIDGIN rules. It should be stressed that this facility should *not* be used anywhere in the grammar in any way that affects the resulting parse. It should be possible to isolate all PIDGIN code that is "semantic" and remove it from the grammar without affecting the overall behavior of the parser for any input in any way. All uses of the following primitives should be restricted to such "semantic" code.

One can embed LISP code inside a grammar rule by simply prefixing any S-expression with "!". To include PIDGIN code inside the LISP code, simply surround the PIDGIN code with "{}". An example of such mixed code is:

If 1st is a vp then !(PRINT {the verb of 1st}).

PIDGIN also provides a means for calling arbitrary LISP functions without leaving the PIDGIN reader. To call a LISP function, simply use the "FORTRAN" function syntax. Thus, the LISP code

(plus 3 4 5)

is equivalent to the PIDGIN code

plus(3, 4, 5)

While the grammar model does not formally include such a mechanism, it is possible to place and retrieve registers on parse nodes. PIDGIN provides primitives for setting and retrieving the values of registers. The command to set a register is

Set the <register> of <node> to <value>.

The phrase whose value is the contents of a register is
the <register> register of <node>.

Traces

See Chapter 5 for a description of the following commands:

Set the binding of <node> to <controlling node>.
the binding of <node>

Attention Shifting

See Chapter 8 for a description of the following command:

Restore the buffer.